

Clase 7

Seguridad y Hardening con Bash

Programacion para Ciberseguridad — Fase II

Scripting de Seguridad Defensiva en Linux — Semana 7

Objetivo de la sesion

- Distinguir monitoreo de hardening y entender por que ambos son necesarios
- Detectar permisos peligrosos (SUID, world-writable) con `find`
- Auditar cuentas de usuario: UID 0 extra, sin contraseña, shells activos
- Aplicar hardening de SSH con `sed` de forma segura y reproducible
- Detectar servicios activos no autorizados con `systemctl`
- Integrar todo en un script de auditoria de seguridad completo

Monitoreo vs. hardening

El script de la sesion anterior

Analizamos `auth.log`, contamos intentos fallidos y generamos alertas por IP. Eso es **monitoreo**: detectar lo que ya ocurrio.

El problema del monitoreo puro

Si el sistema tiene `PermitRootLogin yes`, hay cuentas sin contrasena y servicios innecesarios corriendo — un atacante puede lograrlo **aunque tu estes mirando**.

Hardening: cerrar la brecha antes

Reducir la superficie de ataque: eliminar lo que no hace falta, bloquear lo que deberia estar bloqueado, restringir lo que deberia estar restringido.

Auditoria de permisos peligrosos

```
# Archivos con bit SUID (corren como el propietario, no como quien los llama)
find / -perm -4000 -type f 2>/dev/null

# Archivos con escritura abierta para cualquier usuario
find / -perm -0002 -type f 2>/dev/null | grep -v /proc | grep -v /sys

# Archivos sin propietario valido (senal de cuenta eliminada)
find / -nouser -o -nogroup 2>/dev/null | grep -v /proc
```

Aplicacion en ciberseguridad

Un atacante con acceso limitado busca archivos SUID de root para **escalar privilegios**. Esta busqueda es uno de los primeros pasos en cualquier post-exploitation. Si lo haces tu antes, sabes exactamente que puede aprovechar.

Error comun

Olvidar el 2>/dev/null: sin el, la terminal se llena de "Permission denied" y es imposible leer los resultados.

Auditoria de cuentas de usuario

```
# Cuentas con UID 0 ademas de root (puerta trasera o error grave)
awk -F: '($3 == 0) {print $1}' /etc/passwd

# Cuentas sin contrasena establecida
sudo awk -F: '($2 == "" || $2 == "!") {print $1}' /etc/shadow

# Cuentas con shell interactivo activo (pueden iniciar sesion)
grep -v '/sbin/nologin\|/bin/false' /etc/passwd | awk -F: '{print $1, $7}'
```

Hallazgo	Riesgo
UID 0 adicional	Acceso de superusuario no documentado
Sin contrasena	Acceso sin autenticacion
Shell activo innecesario	Vector de acceso disponible

Hardening de SSH: las tres reglas de oro

```
# 1. Verificar antes de tocar
grep -E "^#?(PermitRootLogin|PasswordAuthentication|MaxAuthTries)" \
    /etc/ssh/sshd_config

# 2. Siempre hacer respaldo con fecha
cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.$(date +%Y%m%d)

# 3. Aplicar con sed
sudo sed -i 's/^#*PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/
    sshd_config
sudo sed -i 's/^#*MaxAuthTries.*/MaxAuthTries 3/' /etc/ssh/sshd_config

sudo systemctl reload ssh
```

Riesgo critico antes de aplicar

Si desactivas `PasswordAuthentication` sin tener una clave publica configurada y **probada**, te bloqueas del servidor. Verifica el acceso por clave primero.

Por que esas tres configuraciones?

Parametro	Por que importa
<code>PermitRootLogin no</code>	Obliga al atacante a conocer tambien el nombre de usuario, no solo la contraseña. Elimina el vector mas atacado.
<code>PasswordAuthentication no</code>	Solo permite autentificacion por clave publica — mucho mas dificil de comprometer que una contraseña.
<code>MaxAuthTries 3</code>	Cierra la conexion despues de 3 intentos, frenando ataques de fuerza bruta aunque sean lentos.

Tip

Estas tres reglas son el minimo. Cambiar el puerto por defecto (22) y usar `AllowUsers` para restringir quien puede conectarse son los siguientes pasos.

Auditoria de servicios activos

```
servicios_esperados=("ssh.service" "cron.service" "rsyslog.service" "ufw.service")

systemctl list-units --type=service --state=running --no-pager --plain | \
  awk '{print $1}' | grep '\.service$' | while read -r svc; do
    esperado=false
    for s in "${servicios_esperados[@]}"; do
      [[ "$svc" == "$s" ]] && esperado=true && break
    done
    [[ "$esperado" == false ]] && echo "REVISAR: $svc"
  done
```

Principio de minimo privilegio aplicado a servicios

Todo servicio activo es un vector potencial. La lista servicios_esperados cambia segun el rol del servidor — el script no.

Script integrador de auditoria de seguridad

```
#!/bin/bash
ROJO='\033[0;31m'; VERDE='\033[0;32m'; RESET='\033[0m'
REPORTE="auditoria_$(date +%Y%m%d_%H%M%S).txt"

alerta() { echo -e "${ROJO}[!] $1${RESET}"; echo "[ALERTA] $1" >> "$REPORTE"; }
ok()      { echo -e "${VERDE}[OK] $1${RESET}"; echo "[OK] $1" >> "$REPORTE"; }

# 1. UID 0 extra
extra=$(awk -F: '($3==0){print $1}' /etc/passwd | grep -v '^root$')
[ -z "$extra" ] && ok "Solo root tiene UID 0" || alerta "UID 0 extra: $extra"

# 2. PermitRootLogin
prl=$(grep -E "^PermitRootLogin" /etc/ssh/sshd_config 2>/dev/null | awk '{print $2}')
[ "$prl" == "no" ] && ok "PermitRootLogin = no" \
    || alerta "PermitRootLogin = ${prl:-sin configurar}"

# 3. Archivos SUID
n=$(find / -perm -4000 -type f 2>/dev/null | wc -l)
echo "[-] Archivos SUID: $n (revisar manualmente)" | tee -a "$REPORTE"

chmod 600 "$REPORTE" && echo "Reporte: $REPORTE"
```

Cierre de sesion y de Fase II

1. **Monitoreo** detecta lo que ya paso; **hardening** evita que pueda pasar
2. SUID, cuentas con UID 0 y SSH mal configurado son los tres vectores mas auditados
3. `sed -i` con respaldo previo es la forma segura de modificar configuraciones
4. Verificar acceso por clave publica **antes** de desactivar contraseñas en SSH
5. El script de auditoria sigue el mismo patron del Blue Team Cookbook de Fase I

Retos: R21 (auth.log → JSON) · R22 (variables + systemctl)

A continuacion: Fase III — Python Ofensivo y Defensivo